

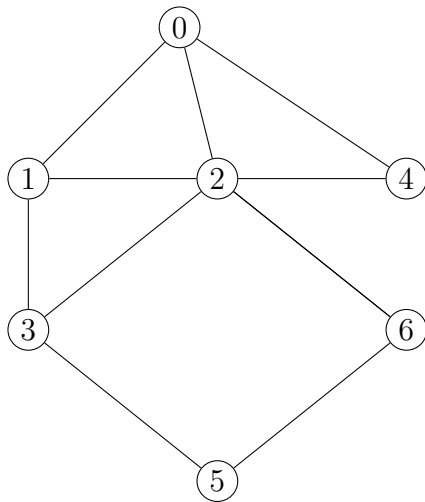
- Correction du TD 5 -

Parcours d'un graphe en largeur : BFS (*Breadth First Search*)

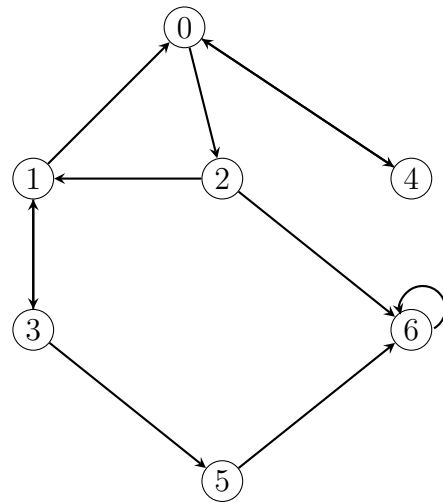
Exercice 1 :

1. On considère les deux graphes ci-dessous :

Graphe G_1 :



Graphe G_2 :



- (a) Pour le graphe G_1 et le sommet source $s = 5$:

Étape initiale	Access=[5]	Avisiter=[5]
Étape n° 1 (visite du 1er sommet : 5)	Access=[6,3,5]	Avisiter= [6,3]
Étape n° 2 (visite du 2ième sommet : 3)	Access=[2,1,6,3,5]	Avisiter=[2,1,6]
Étape n° 3 (visite du 3ième sommet : 6)	Access=[2,1,6,3,5]	Avisiter=[2,1]
Étape n° 4 (visite du 4ième sommet : 1)	Access=[0,2,1,6,3,5]	Avisiter=[0,2]
Étape n° 5 (visite du 5ième sommet : 2)	Access=[4,0,2,1,6,3,5]	Avisiter=[4,0]
Étape n° 6 (visite du 6ième sommet : 0)	Access=[4,0,2,1,6,3,5]	Avisiter=[4]
Étape n° 7 (visite du 7ième sommet : 4)	Access=[4,0,2,1,6,3,5]	Avisiter=[]

Pour le graphe G_1 et le sommet source $s = 2$:

Étape initiale	Access=[2]	Avisiter=[2]
Étape n° 1 (visite du 1er sommet : 2)	Access=[6,4,3,1,0,2]	Avisiter=[6,4,3,1,0]
Étape n° 2 (visite du 2ème sommet : 0)	Access=[6,4,3,1,0,2]	Avisiter=[6,4,3,1]
Étape n° 3 (visite du 3ème sommet : 1)	Access=[6,4,3,1,0,2]	Avisiter=[6,4,3]
Étape n° 4 (visite du 4ème sommet : 3)	Access=[5,6,4,3,1,0,2]	Avisiter=[5,6,4]
Étape n° 5 (visite du 5ème sommet : 4)	Access=[5,6,4,3,1,0,2]	Avisiter=[5,6]
Étape n° 6 (visite du 6ème sommet : 6)	Access=[5,6,4,3,1,0,2]	Avisiter=[5]
Étape n° 7 (visite du 7ème sommet : 5)	Access=[5,6,4,3,1,0,2]	Avisiter=[]

Pour le graphe G_1 et le sommet source $s = 1$:

Étape initiale	Access=[1]	Avisiter=[1]
Étape n° 1 (visite du 1er sommet : 1)	Access=[3,2,0,1]	Avisiter=[3,2,0]
Étape n° 2 (visite du 2ème sommet : 0)	Access=[4,3,2,0,1]	Avisiter=[4,3,2]
Étape n° 3 (visite du 3ème sommet : 2)	Access=[6,4,3,2,0,1]	Avisiter=[6,4,3]
Étape n° 4 (visite du 4ème sommet : 3)	Access=[5,6,4,3,2,0,1]	Avisiter=[5,6,4]
Étape n° 5 (visite du 5ème sommet : 4)	Access=[5,6,4,3,2,0,1]	Avisiter=[5,6]
Étape n° 6 (visite du 6ème sommet : 6)	Access=[5,6,4,3,2,0,1]	Avisiter=[5]
Étape n° 7 (visite du 7ème sommet : 5)	Access=[5,6,4,3,2,0,1]	Avisiter=[]

Pour le graphe G_2 et le sommet source $s = 5$:

Étape initiale	Access=[5]	Avisiter=[5]
Étape n° 1 (visite du 1er sommet : 5)	Access=[6, 5]	Avisiter=[6]
Étape n° 2 (visite du 2ème sommet : 6)	Access=[6,5]	Avisiter=[]

Pour le graphe G_2 et le sommet source $s = 2$:

Étape initiale	Access=[2]	Avisiter=[2]
Étape n° 1 (visite du 1er sommet : 2)	Access=[6,1,2]	Avisiter=[6,1]
Étape n° 2 (visite du 2ème sommet : 1)	Access=[3,0,6 ,1,2]	Avisiter=[3,0,6]
Étape n° 3 (visite du 3ème sommet : 6)	Access=[3,0,6 ,1,2]	Avisiter=[3,0]
Étape n° 4 (visite du 4ème sommet : 0)	Access=[4,3,0,6 ,1,2]	Avisiter=[4,3]
Étape n° 5 (visite du 5ème sommet : 3)	Access=[5,4,3,0,6 ,1,2]	Avisiter=[5,4]
Étape n° 6 (visite du 6ème sommet : 4)	Access=[5,4,3,0,6 ,1,2]	Avisiter=[5]
Étape n° 7 (visite du 7ème sommet : 5)	Access=[5,4,3,0,6 ,1,2]	Avisiter=[]

Pour le graphe G_2 et le sommet source $s = 1$:

Étape initiale	Access=[1]	Avisiter=[1]
Étape n° 1 (visite du 1er sommet : 1)	Access=[3,0,1]	Avisiter=[3, 0]
Étape n° 2 (visite du 2ème sommet : 0)	Access=[4,2,3,0,1]	Avisiter=[4,2,3]
Étape n° 3 (visite du 3ème sommet : 3)	Access=[5,4,2,3,0,1]	Avisiter=[5,4,2]
Étape n° 4 (visite du 4ème sommet : 2)	Access=[6,5,4,2,3,0,1]	Avisiter=[6, 5,4]
Étape n° 5 (visite du 5ème sommet : 4)	Access=[6,5,4,2,3,0,1]	Avisiter=[6,5]
Étape n° 6 (visite du 6ème sommet : 5)	Access=[6,5,4,2,3,0,1]	Avisiter=[6]
Étape n° 7 (visite du 7ème sommet : 6)	Access=[6,5,4,2,3,0,1]	Avisiter=[]

- (b) Les commandes ci-dessous, définissant le graphe non orienté G_1 , et utilisant le sommet source $s = 5$:

```

D1={0:[1,2,4],1:[0,2,3],2:[0,1,3,4,6],\
    3:[1,2,5],4:[0,2],5:[3,6],6:[2,5]}

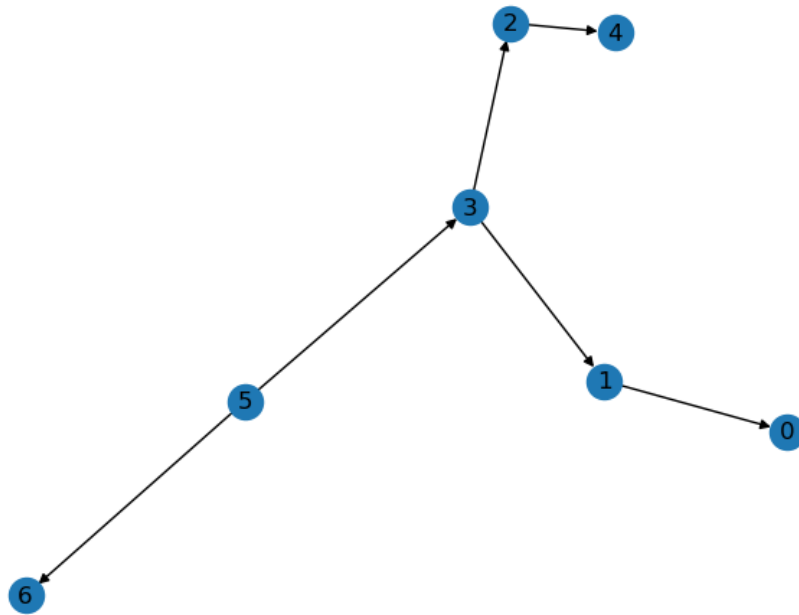
G1=nx.Graph(D1)

plt.clf()
B1=nx.bfs_tree(G1,source=5)
nx.draw(B1,with_labels=True)

plt.show()

```

renvoient la fenêtre graphique suivante :



Le graphe donné en Python montre bien qu'on obtient dans ce cas :

- à l'étage 0 : 5,
- à l'étage 1 : 6 et 3,
- à l'étage 2 : 1 et 2,
- à l'étage 3 : 0 et 4.

2. (a) Un exemple de fonction :

```

def Parcours(G,s):
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[]:
        u=Avisiter.pop()
        L=G[u]
        for k in L:
            if not (k in Access):
                Avisiter.insert(0,k)
                Access.append(k)
    return(Access)

```

(b) Soit $n \in \mathbb{N} - \{0, 1\}$.

On sait que dans le graphe complet K_n , chaque sommet est relié aux $n - 1$ autres sommets.

Représentation du graphe complet K_5 en Python :

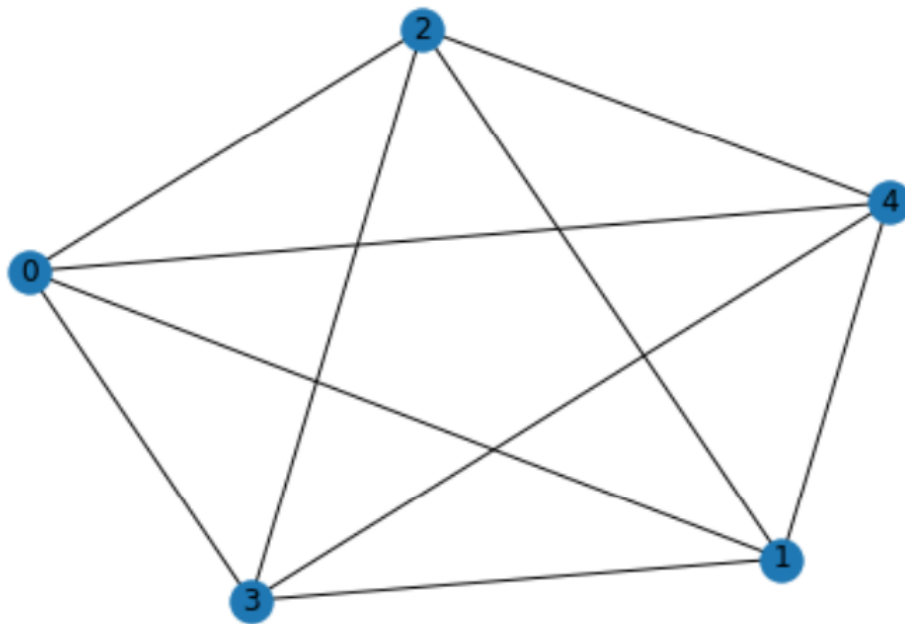
Les commandes ci-dessous

```

G=nx.complete_graph(5)
nx.draw(G,with_labels=True)
plt.show()

```

ont donné la fenêtre graphique ci-dessous :



Dans un parcours en largeur du graphe complet K_n , dès le premier tour de l'algorithme, tous les sommets sont trouvés, car ils sont tous voisins du sommet source. On peut donc ajouter un test d'arrêt qui stoppe l'algorithme dès que tous les sommets ont déjà été découverts.

```
def Parcours(G,s):
    n=len(G)
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[] and len(Access)<n:
        u=Avisiter.pop()
        L=G[u]
        for k in L:
            if not (k in Access):
                Avisiter.insert(0,k)
                Access.append(k)
    return(Access)
```

(c)

```
def Parcours2(M,s):
    n=len(M)
    Access=[s]
    Avisiter=[s]
    while Avisiter!=[] and len(Access)<n:
        u=Avisiter.pop()
        for j in range(n):
            if M[u,j]==1 and not (j in Access):
                Avisiter.insert(0,j)
                Access.append(j)
    return(Access)
```

3. Un exemple de fonction récursive :

```
def BFSrec(G, Access, Avisiter):  
    if Avisiter==[]:  
        return(Access)  
    else:  
        s=Avisiter.pop()  
        L=G[s]  
        for k in L:  
            if not (k in Access):  
                Avisiter.insert(0,k)  
                Access.append(k)  
        return(BFSrec(G, Access, Avisiter))
```

Par exemple, pour obtenir la liste des sommets accessibles depuis le sommet 5 du graphe G_1 , on peut écrire :

```
G={0:[1,2,4],1:[0,2,3],2:[0,1,3,4,6],\  
3:[1,2,5],4:[0,2],5:[3,6],6:[2,5]}  
  
Acc=[5]  
  
Avis=[5]  
  
BFSrec(G, Acc, Avis)
```

Exercice 2 : Applications de l'algorithme BFS

1. Etude de la connexité d'un graphe non orienté

(a) Soit G un graphe non orienté.

$\implies$

Supposons que G est connexe.

Soit s un sommet de G .

Comme G est connexe, tout sommet de G distinct de s est accessible depuis le sommet s .

$\impliedby$

Supposons qu'il existe un sommet s de G tel que pour tout sommet t de G tel que $t \neq s$, il existe une chaîne dans G reliant s à t .

Soient u et v deux sommets distincts de G .

Si $u = s$, alors il existe bien une chaîne reliant u à v , d'après notre hypothèse.

De même, si $v = s$, il existe une chaîne reliant v à u , et donc de u à v , puisque G n'est pas orienté.

Supposons maintenant que $u \neq s$ et $v \neq s$.

Par hypothèse, il existe une chaîne $c_1 = (s, u_1, \dots, u_p)$, avec $u_p = u$ reliant s à u . Et de même, il existe une chaîne $c_2 = (s, v_1, \dots, v_q)$, avec $v_q = v$, reliant s à v .

Alors, la concaténation $c = (u_p, u_{p-1}, \dots, u_1, s, v_1, \dots, v_q)$ est une chaîne dans G qui relie u et v .

On en déduit que le graphe G est bien connexe.

L'équivalence est bien établie.

(b) Soit G un graphe non orienté.

Par définition de la connexité,

→ si pour tout sommet s de G , tout sommet de G , distinct de s , est accessible depuis s , alors G est connexe,

→ si il existe un sommet de G , distinct de s , et non accessible depuis s , alors G n'est pas connexe.

Proposition de fonction :

```
def Est_connexe(G):
    L=G.keys()
    n=len(L)
    L=list(L)
    s=L[0]
    Acc=[s]
    Avoir=[s]
    while Avoir!=[]:
        u=Avoir.pop()
        voisin=G[u]
        for k in voisin:
            if not(k in Acc):
                Acc.append(k)
                Avoir.insert(0,k)
    if len(Acc)==n:
        return('Est connexe')
    else:
        return('Non connexe')
```

- (c) L'algorithme de parcours en largeur, partant d'un sommet s de G , donne l'ensemble des sommets de G accessibles depuis le sommet source s , c'est-à-dire la composante connexe de s .

Proposition de fonction :

```
def Nbconnexe(G):
    L=G.keys()
    n=len(L)
    L=list(L)
    s=L.pop()
    Acc=[s]
    Avoir=[s]
    nbC=1
    while len(Acc)<n:
        if Avoir==[]:
            nbC=nbC+1
            s=L.pop()
            Acc.append(s)
            Avoir=[s]
        u=Avoir.pop()
        voisin=G[u]
        for k in voisin:
            if not(k in Acc):
                Acc.append(k)
                Avoir.insert(0,k)
                L.remove(k)
    return(nbC)
```

2. Recherche des cycles (ou circuits) d'un graphe

(a)

```
def circ_source(M,s):
    n=len(M)
    Vu=[s]
    Atraiter=[s]
    while Atraiter!=[]:
        u=Atraiter.pop()
        if M[u,s]>0:
            return('True')
        else:
            for i in range(n):
                if M[u,i]>0 and not(i in Vu):
                    Atraiter.insert(0,i)
                    Vu.append(i)
    return('False')
```


(b)

```
def circuit (M) :  
    n=len(M)  
    for s in range(n):  
        if circ_source(M,s)=='True':  
            return('True')  
    return('False')
```

3. Recherche des triangles d'un graphe

(a)

```
def triangle_source(G,s):  
    Voisin=G[s]  
    Voisin=list(Voisin)  
    if len(Voisin)<=1:  
        return(0)  
    else:  
        Nb=0  
        d=len(Voisin)  
        for i in range(d-1):  
            u=Voisin[i]  
            for j in range(i+1,d):  
                v=Voisin[j]  
                if v in G[u]:  
                    Nb=Nb+1  
        return(Nb)
```

(b)

```
def triangle(G):  
    N=0  
    for s in G.keys():  
        N=N+triangle_source(G,s)  
    return(N/3)
```

4. Recherche d'une partition des sommets pour un graphe biparti

(a) Pour définir les trois matrices M_i , il suffit de taper les commandes :

```
M1=np.array  
    ([[0,1,0,1],[1,0,1,0],[0,1,0,1],[1,0,1,0]])  
M2=np.array([[0,1,0,0],[  
    1,0,0,0],[0,0,0,1],[0,0,1,0]])  
M3=np.array([[0,1,1,1,1],[1,0,1,1,1],[1,1,0,1,1],\  
    [1,1,1,0,1],[1,1,1,1,0]])
```

- Graphiquement, le graphe G_1 est le « carré » de sommets 0, 1, 2, 3. Autrement dit, le graphe G_1 est le cycle (0, 1, 2, 3, 0).
- Graphiquement, le graphe G_2 est le graphe composé de deux arêtes : 0 – 1 et 2 – 3.

- Le graphe G_3 est le graphe complet d'ordre 5.

(b) Après la commande `print(mystere(M1))`, l'écran affiche :

([1, 3], [0, 2])

Après la commande `print(mystere(M2))`, l'écran affiche :

False

Après la commande `print(mystere(M3))`, l'écran affiche :

False

(c) Soit $n \in \mathbb{N}^*$.

Notons G le cycle $(0, 1, 2, \dots, n-1, 0)$.

Pour tout $i \in \llbracket 1, n-2 \rrbracket$, i a pour voisins $i-1$ et $i+1$. Et 0 a pour voisins 1 et $n-1$, $n-1$ a pour voisin $n-2$ et 0.

La matrice d'adjacences de G , notée $M = (m_{i,j})_{0 \leq i,j \leq n-1}$, est définie par :

$$M = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 & 1 \\ 1 & 0 & \ddots & \ddots & & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & & \ddots & \ddots & \ddots & 1 \\ 1 & 0 & \cdots & 0 & 1 & 0 \end{pmatrix}$$

```
def mat(n):
    M=np.zeros([n,n])
    for i in range(n-1):
        M[i,i+1]=1
    # On traite pour M[i,i+1] tous les termes au dessus
    # de la diagonale
        M[i+1,i]=1
    # On traite pour M[i+1,i] tous les termes en dessous
    # de la diagonale
        M[0,n-1]=1
        M[n-1,0]=1
    return(M)
```

On remarque que pour $n = 2k$ pair, la commande `mystere(mat(n))` renvoie deux listes : $[1, 3, 5, \dots, 2k-1]$ et $[0, 2, 4, \dots, 2k]$.

Et pour $n = 2k + 1$ impair, la commande `mystere(mat(n))` renvoie : **False**.

(d) La fonction `mystere` appliquée à une matrice M représentant un graphe G non orienté renvoie **False** si le graphe G n'est pas biparti, et renvoie lorsque le graphe G est biparti, une partition (S_1, S_2) de ses sommets telle que toute arête de G a une de ses extrémité dans S_1 et l'autre dans S_2 .

La fonction `mystere` suit l'algorithme de parcours en largeur et utilise en plus une liste notée **Etage**.

Lorsqu'un nouveau sommet s est exploré :

- Si ce sommet découle du début de l'exploration (début de l'algorithme, ou découverte d'une nouvelle composante connexe), on affecte à s la valeur 1 : dans la liste **Etage**, la composante représentant s (situé au rang s) prend la valeur 1 ;

Si ce sommet s provient d'un voisin déjà étudié, son étage d'affectation est déjà défini (1 ou -1).

- On étudie alors ce nouveau sommet s : on en cherche les voisins à l'aide des commandes

```
L=M[s, :]  
    for u in range(n):  
        if L[u]==1:
```

Lorsque u est un voisin de s :

- soit ce voisin u a déjà été découvert lors du parcours en largeur et l'étage affecté à u est le même que celui de s : on ne peut plus tracer une arête $\{u, s\}$ avec une extrémité dans chacun des étages. On affecte alors à B la valeur **False** : la fonction renverra alors **False** et le graphe G est non biparti.
- soit ce voisin u a déjà été découvert lors du parcours en largeur et l'étage affecté à u est l'opposé de celui de s : l'arête $\{u, s\}$ de G a une extrémité dans chacun des étages. On continue l'étude des autres sommets de G : le graphe G est peut-être biparti.
- soit ce voisin u n'a pas encore été découvert lors du parcours en largeur et dans ce cas, on affecte comme étage à u , l'opposé de celui de s : l'arête $\{u, s\}$ de G a une extrémité dans chacun des étages. On continue l'étude des autres sommets de G : le graphe G est peut-être biparti.

5. Algorithme du plus court chemin

- (a) *Dans un parcours en largeur, on visite d'abord le sommet source, puis la génération n° 1 des sommets qui sont à distance 1 (en nombre d'arêtes ou arcs), puis la génération n° 2 des sommets qui sont à distance 2, etc. Quand on visite un sommet, nommé ici le point d'arrivée, le numéro de sa génération représente le nombre d'arêtes qui le séparent du point de départ. Autrement dit, un parcours en largeur à partir du sommet source permet de trouver un plus court chemin vers le sommet d'arrivée en nombre d'étapes (c'est-à-dire d'arêtes ou d'arcs traversés). Si on prend la précaution de noter (dans une liste) le prédécesseur (ou « père ») de chaque génération de sommets, il suffit de remonter de père en père pour reconstituer le chemin d'accès du point d'arrivée.*

```

inf=float('inf')

def distance (G,s) :
    n=len(G)
    u=s
    Vu=[u]
    Atraiter=[u]
    dist=[inf for k in range(n)]
    parent=dist
    dist[u]=0
    parent[u]=u
    while Atraiter!=[]:
        u=Atraiter.pop()
        L=G[u]
        for v in L:
            if not(v in Vu):
                Vu.append(v)
                Atraiter.insert(0,v)
                dist[v]=dist[u]+1
                parent[v]=u
    return(dist,parent)

```

(b)

```

def chemin(G,s,t) :
    dist,parent=distance(G,s)
    if dist[t]==inf:
        return(inf)
    else :
        c=[]
        v=t
        while v!=s:
            c.insert(0,v)
            v=parent[v]
        c.insert(0,s)
        return(c)

```
